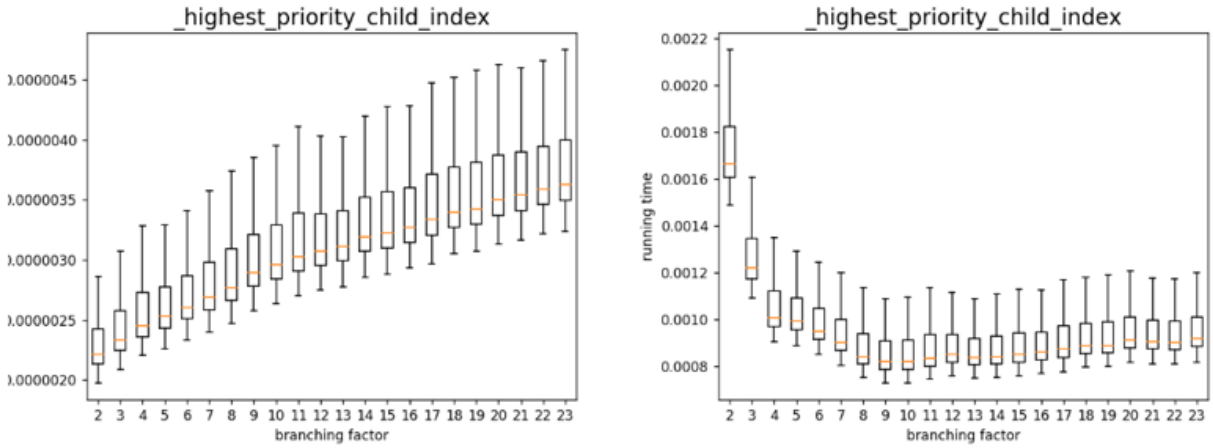


**Figure 2.24 (A) The distribution of recorded running times per-call for `_highest_priority_child`.
(B) The distribution of cumulative running times for `_highest_priority_child`.**

You might then wonder: why then `_push_down` doesn't have the same trend? Remember that while the running time for `_highest_priority_child` is `O(D)`, in particular with `D` comparisons, `_push_down` performs (at most) `logD(n)` swap sand `D*logD(n)` comparisons, because it calls `_highest_priority_child` at most `D` times.

The largest it is the branching factor `D`, the fewer calls are made to `_highest_priority_child`: this becomes apparent if, instead of plotting the running time per-call for `_highest_priority_child`, we use the total cumulative time (the sum of all calls to each method), as shown in figure 2.24.B: there we can see again how this composite function, `f(D) = D*logD(n)`, has a minimum at `D==9`.

2.10.3 The Mystery with Heapify